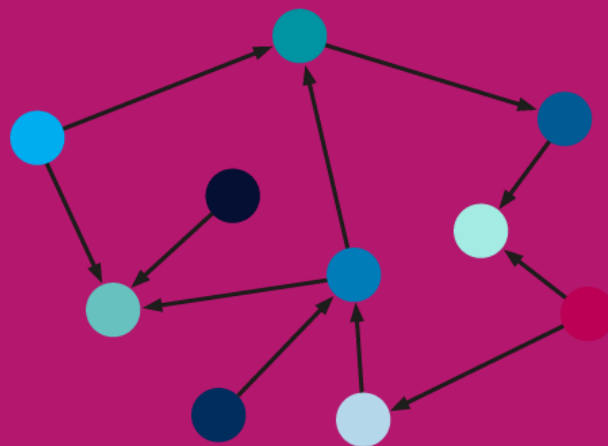
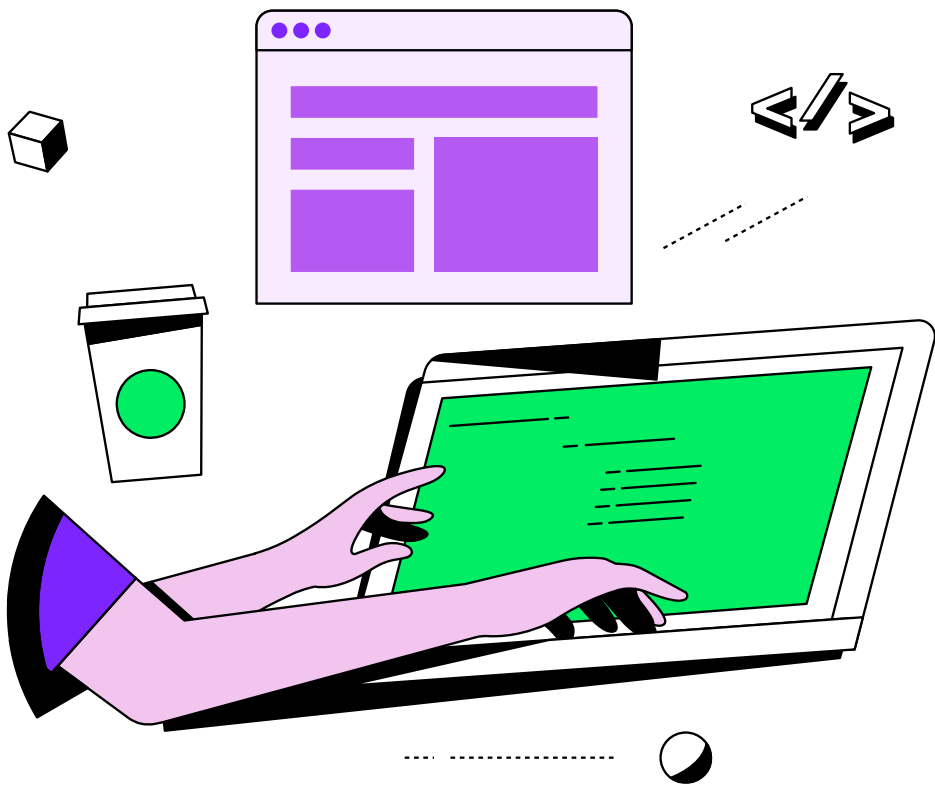




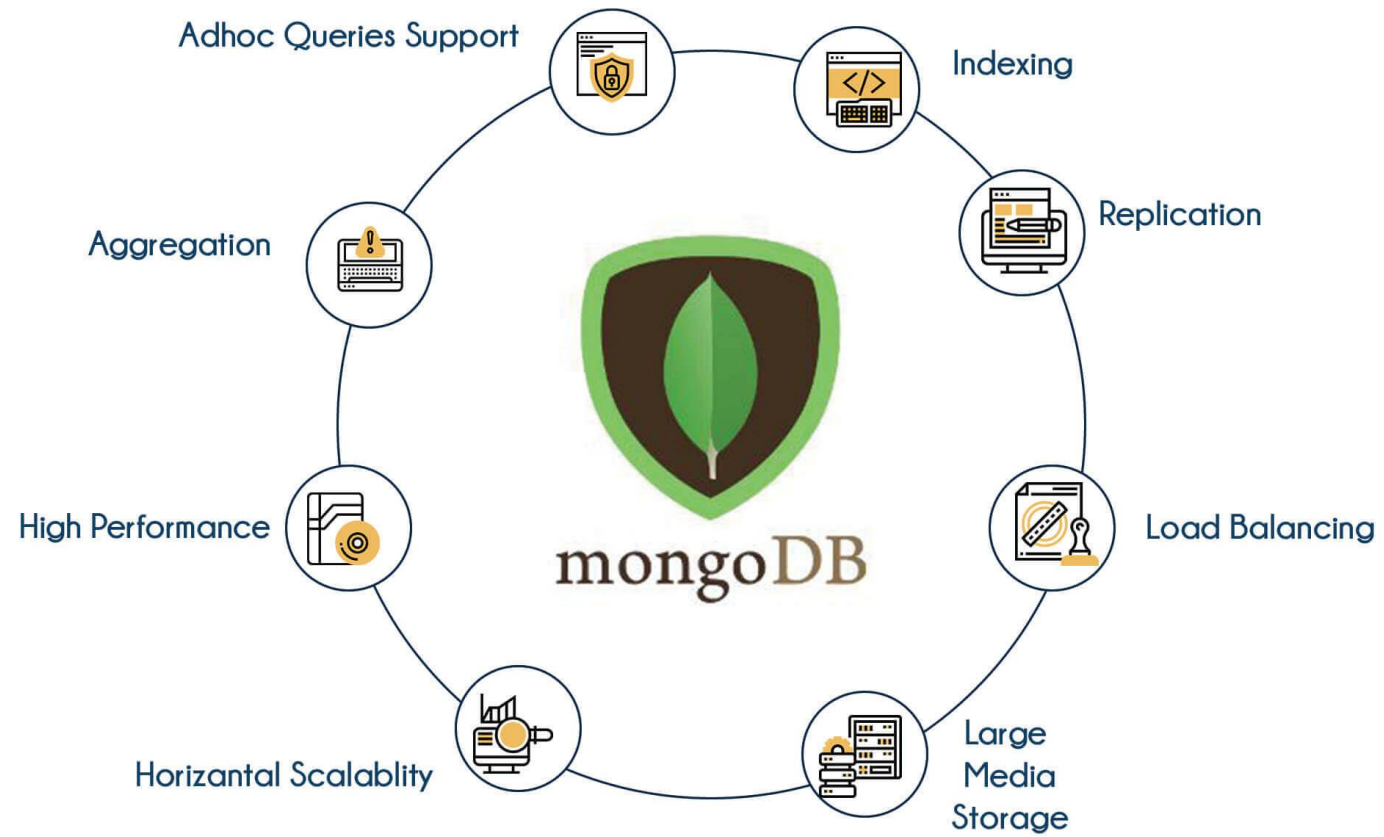
MongoDB

Основні можливості

Жишкович Андрій Володимирович



- ❖ Реплікації
- ❖ Індокси
- ❖ Типи даних
- ❖ Агрегація
- ❖ Пов'язування документів
- ❖ Ad-hoc запити



Реплікації в MongoDB

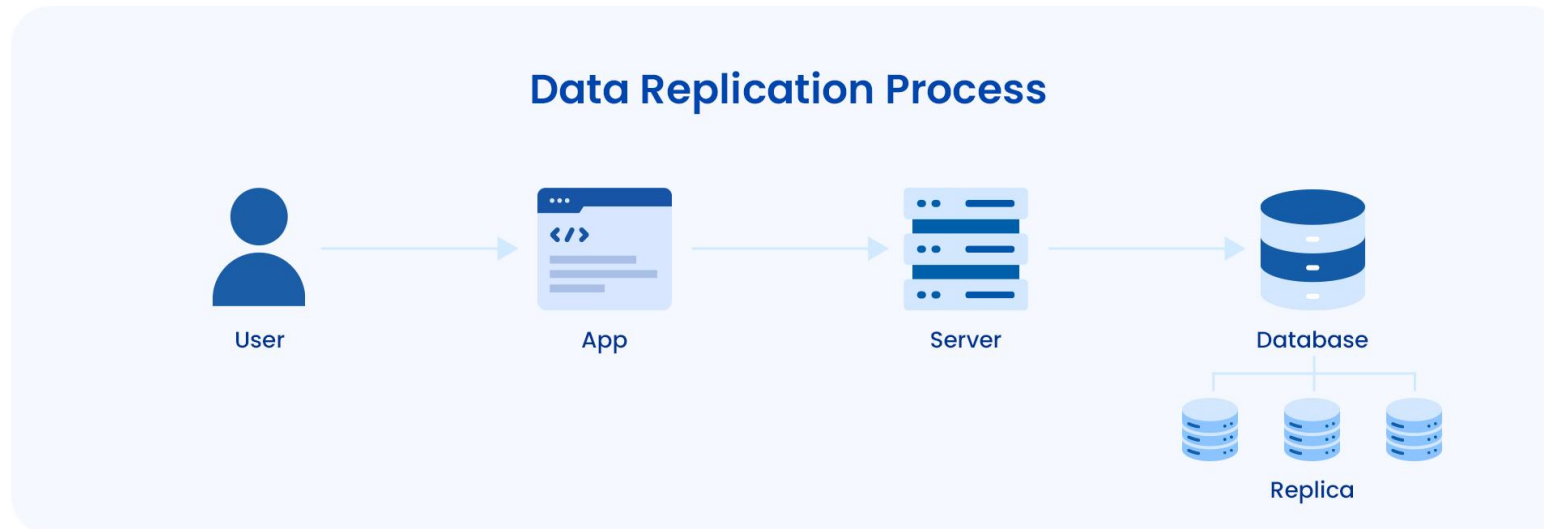
Реплікації

Реплікація — це процес підтримки кількох копій одних і тих самих даних на різних серверах.

Основна мета реплікації - підвищити доступність даних, розподілити робоче навантаження та підвищити загальну продуктивність системи бази даних.

Реплікація забезпечує високу надлишковість (декілька копій) і високу доступність даних шляхом реплікації одного набору даних на кількох вузлах.

У разі виходу сервера з ладу через апаратний збій або з іншої причини, дані все ще доступні на іншому вузлі, і додаток може залишатися онлайн.

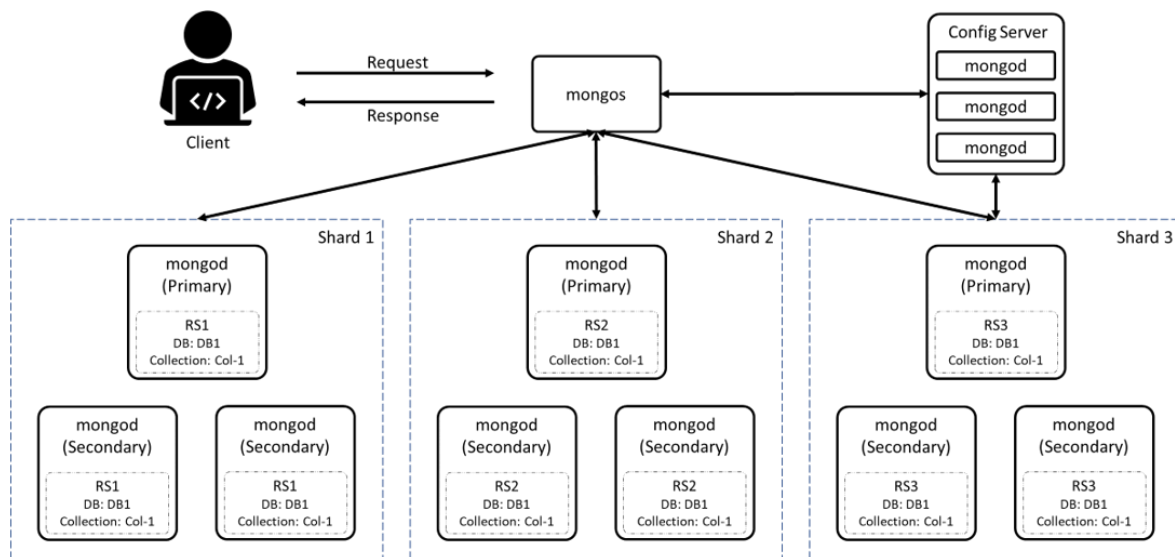


Реплікація в MongoDB

Mongos – це інтерфейс між **клієнтом** і **шардинг-кластерами**. Він підключається до **конфігураційних серверів**, щоб визначити, який **шард** повинен відповідати на запит.

Шард – це розподіл даних між кількома машинами.

Кожен шард розгортається як **набір реплік**, що забезпечує автоматичне перемикання у разі відмови і забезпечує реплікацію даних для підвищення надійності та доступності.



MongoDB досягає реплікації за допомогою **набору реплік**.

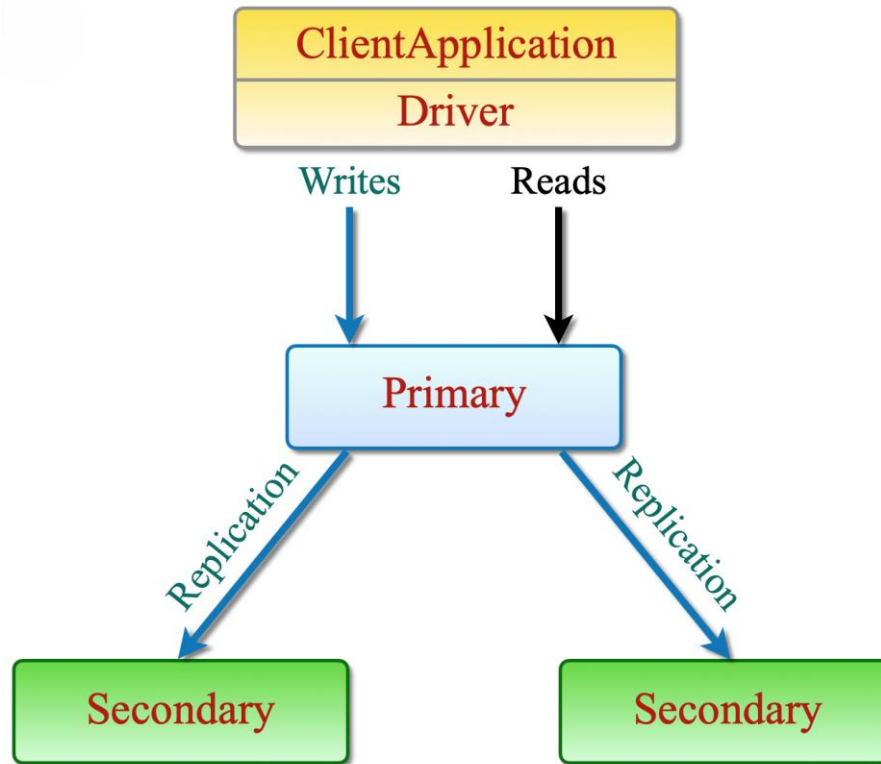
Набір реплік (replica set) — це група екземплярів **mongod**, які містять однаковий набір даних.

У наборі реплік один вузол є основним, який здійснює всі операції запису.

Усі інші екземпляри (вторинні) асинхронно копіюють дані з основного, щоб мати однаковий набір даних.

Набір реплік може мати лише один основний вузол.

Реплікація в MongoDB – типова діаграма реплікації



Типова діаграма реплікації MongoDB, у якій клієнтська програма завжди взаємодіє з основним вузлом, а основний вузол потім реплікує дані на вторинні вузли.

Реплікація в MongoDB – “вибори”

Рекомендується мати щонайменше три процеси mongod у наборі реплік для забезпечення стабільності.

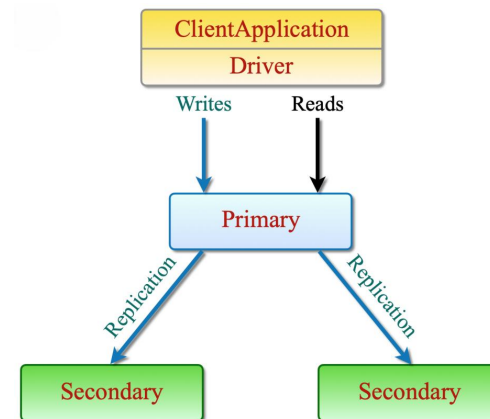
Вузли обмінюються повідомленнями про стан (heartbeat) для перевірки доступності один одного.

Якщо основний стає недоступним, один з вторинних (або адміністратор вручну) ініціює вибори нового основного, і один з вторинних тоді стає новим основним.

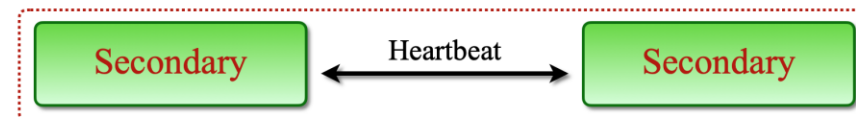
Це забезпечує надлишковість і високу доступність, оскільки під час відмови основного, один з вторинних може стати новим основним через “вибори”.

Після відновлення несправного вузла він знову приєднується до набору реплік і працює як вторинний вузол.

Загалом рекомендується мати непарну кількість набору реплік, які мають право “голосу на виборах”, оскільки це робить “вибори” більш схильними до успішного результату.



Election for New Primary



New Primary Elected

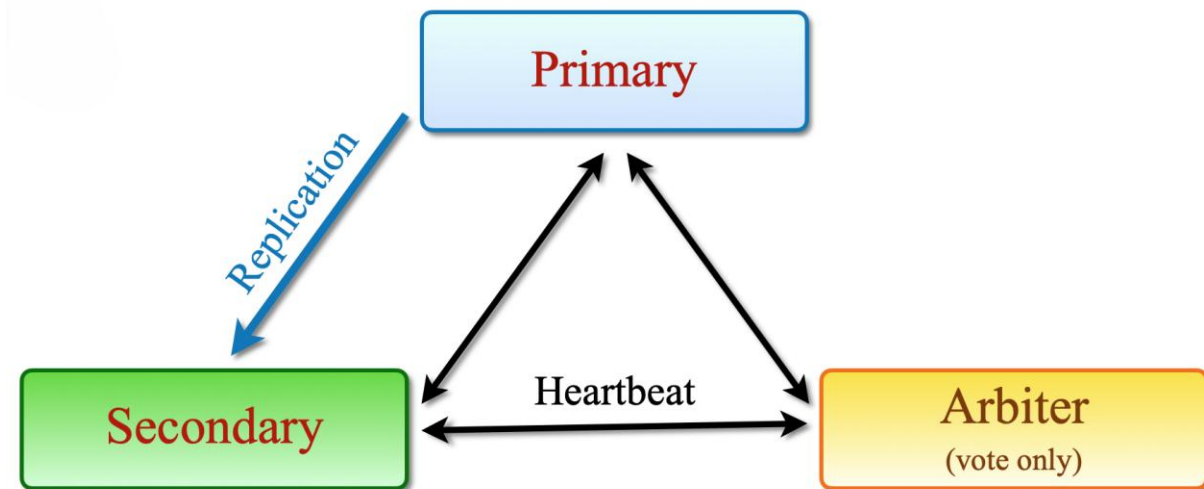


Реплікація в MongoDB – арбітр

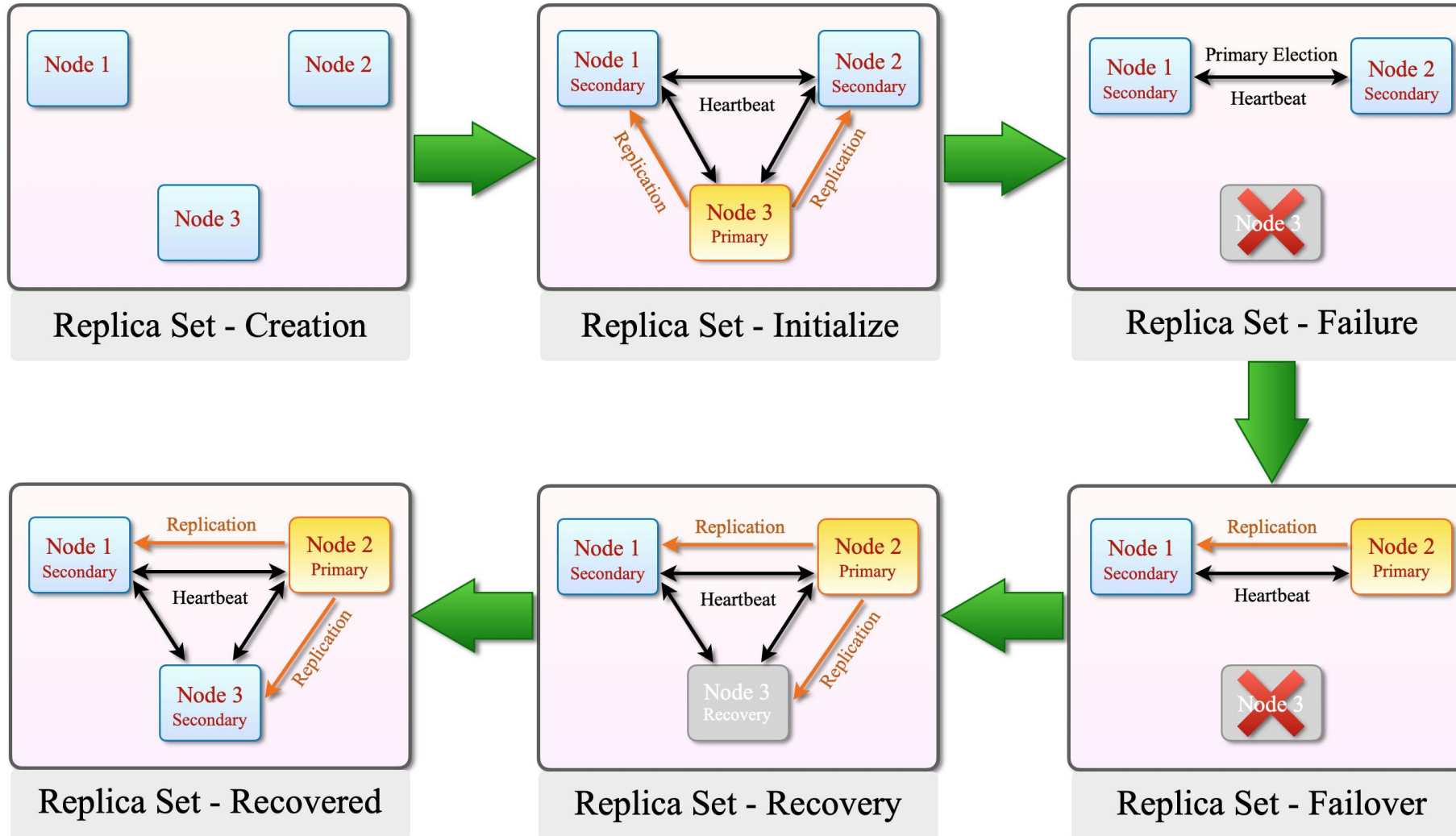
В деяких випадках (наприклад, у вас є основний і вторинний вузли, але через обмеження бюджету ви не можете додати ще один вторинний вузол), ви можете додати інстанцію `mongod` до набору реплік як арбітра.

Арбітр — це спеціальний тип вузла в наборі реплік MongoDB, який використовується для забезпечення кворуму під час виборів основного вузла. Арбітр не зберігає копію даних і не може стати основним вузлом, але він бере участь у виборах і має один голос.

Арбітр завжди буде арбітром, тоді як основний вузол може знизити свій статус і стати вторинним, а вторинний вузол може стати основним під час виборів.



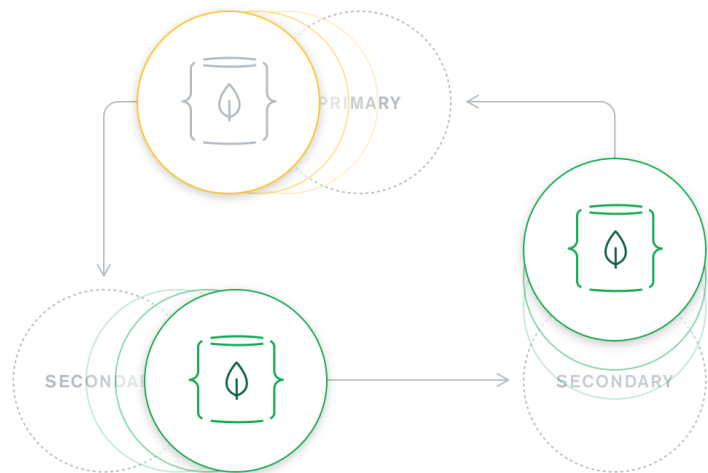
Життєвий цикл “набору реплік” в MongoDB



Декілька загальних кроків процесу реплікації в MongoDB:

- Усі запити на запис від додатка надходять до основного вузла.
- Основний вузол записує всі зміни в журнал операцій, також відомий як oplog.
- Тепер відбувається асинхронна реплікація.
- Вторинний вузол читає журнал операцій основного вузла і застосовує всі ці зміни до своїх наборів даних.
- У разі відмови основного вузла, відповідний вторинний вузол обирається новим основним вузлом.
- Після завершення процесу відновлення вузол приєднується до набору реплік і працює як вторинний вузол.

Основні види реплікації в MongoDB



MongoDB використовує реплікацію на основі наборів реплік для забезпечення високої доступності та надійності даних.

Основні види реплікації, які підтримуються в MongoDB:

Асинхронна реплікація

Це основний метод реплікації в MongoDB. Основний вузол приймає всі операції запису, а потім ці зміни асинхронно реплікуються на вторинні вузли.

Це забезпечує швидке виконання записів, оскільки основний вузол не чекає на підтвердження від вторинних вузлів перед завершенням операції.

Реплікація з відкладеною затримкою (Delayed Replication)

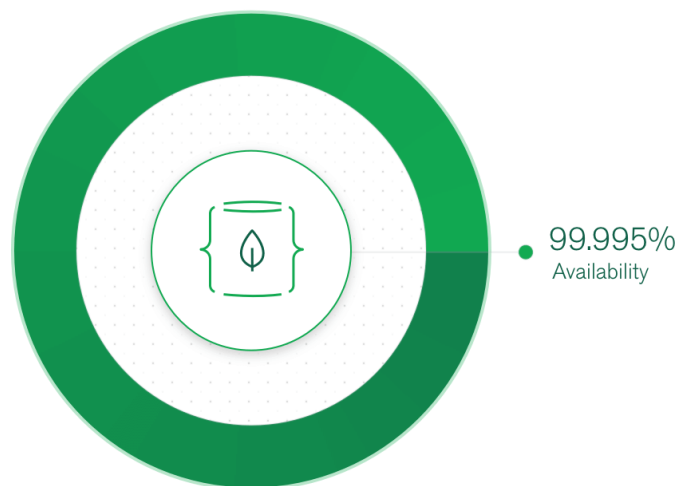
MongoDB дозволяє налаштувати вторинні вузли так, щоб вони реплікували дані з певною затримкою.

Це може бути корисно для захисту від помилкових дій, таких як випадкове видалення даних, оскільки адміністратори мають час для відновлення даних з вторинних вузлів, які ще не встигли реплікувати зміни.

Гібридна реплікація

MongoDB дозволяє налаштувати деякі вторинні вузли для синхронної реплікації, в той час як інші вузли можуть використовувати асинхронну реплікацію.

Це може бути корисно для забезпечення балансу між продуктивністю та узгодженістю даних.



Основні функції набору реплік

Кластер із N вузлів:

Набір реплік може мати будь-яку кількість вузлів, рекомендується непарна кількість для стабільності кворуму.

Будь-який один вузол може бути основним:

Один вузол обробляє всі записи, і може змінюватися через вибори.

Усі операції запису переходять до основного:

Запити на запис спрямовуються до основного вузла, який поширює зміни до вторинних вузлів.

Автоматичне перемикання після відмови:

Під час відмови основного вузла автоматично обирається новий основний вузол.

Автоматичне відновлення

Вторинні вузли синхронізуються з основним і можуть швидко відновити його функції.

Консенсусні вибори первинних

Новий основний вузол обирається більшістю голосів вузлів, забезпечуючи стабільність.

Ці функції набору реплік забезпечують високу доступність та стійкість MongoDB до відмов, роблячи його ідеальним рішенням для критично важливих застосунків, де вимоги до надійності та доступності даних є високими.

Індекси в MongoDB

Що таке індекси?

München (0 89)		Banz-Barn	135
9 82 94 43	Bargställe-Franke Sölke u. Franke-Torsten	69 37 04 76	I
0178 2 33 38 97	Barckhausen-Haasem Tannenheide-28	69 32 94 54	II
6 41 11 17	Barl Pomak Tazvzet Langobühner-17	62 14 61 06	III
64 95 79 25	Barlenth Hertha Hertling-8	64 54 01	IV
6 42 24 31	Baric Anita	30 00 46 70	V
38 32 97 00	- Dragica Mitbertshofer-16	3 59 82 68	VI
37 49 73 58	Ivan Mitbertshofer-44	3 59 85 62	VII
	Barisovic Mato Tutzinger-10	7 69 26 44	VIII
	Barich Christian Architekten	47 08 49 27	IX
3 59 45 23	Lucile-Grahe-27	47 08 49 25	A
53 81 92 30	Lucile-Grahe-27	47 08 49 25	B
0170 1 81 08 21	Barlenth Hertha Hertling-8	64 54 01	C
3 29 17 08	Barlenth Hertha Hertling-8	64 54 01	D
9 28 05 - 0	Barlenth Hertha Hertling-8	64 54 01	E
41 90 10 45	Barlenth Hertha Hertling-8	64 54 01	F
98 57 88	Barlenth Hertha Hertling-8	64 54 01	G
79 67 52	Barlenth Hertha Hertling-8	64 54 01	H
2 71 12 73	Barlenth Hertha Hertling-8	64 54 01	I
2 71 12 73	Barlenth Hertha Hertling-8	64 54 01	J
23 07 65 55	Barlenth Hertha Hertling-8	64 54 01	K
3 08 23 38	Barlenth Hertha Hertling-8	64 54 01	L
6 12 23 38	Barlenth Hertha Hertling-8	64 54 01	M
94 38 57 05	Barlenth Hertha Hertling-8	64 54 01	N
21 96 16 17	Barlenth Hertha Hertling-8	64 54 01	
0170 3 40 37 79	Barlenth Hertha Hertling-8	64 54 01	
4 48 27 26	Barlenth Hertha Hertling-8	64 54 01	
7 00 37 99	Barlenth Hertha Hertling-8	64 54 01	
0178 8 60 16 83	Barlenth Hertha Hertling-8	64 54 01	
75 07 53 63	Barlenth Hertha Hertling-8	64 54 01	
44 23 77 62	Barlenth Hertha Hertling-8	64 54 01	
12 28 19 67	Barlenth Hertha Hertling-8	64 54 01	
93 61 56	Barlenth Hertha Hertling-8	64 54 01	
76 56 08	Barlenth Hertha Hertling-8	64 54 01	
2 71 26 54	Barlenth Hertha Hertling-8	64 54 01	
0171 2 14 62 99	Barlenth Hertha Hertling-8	64 54 01	
32 38 68 79	Barlenth Hertha Hertling-8	64 54 01	
3 20 14 49	Barlenth Hertha Hertling-8	64 54 01	
32 38 68 79	Barlenth Hertha Hertling-8	64 54 01	
38 80 87 07	Barlenth Hertha Hertling-8	64 54 01	
6 13 23 92	Barlenth Hertha Hertling-8	64 54 01	
58 76 54	Barlenth Hertha Hertling-8	64 54 01	
91 68 71	Barlenth Hertha Hertling-8	64 54 01	
6 11 41 47	Barlenth Hertha Hertling-8	64 54 01	
77 97 31	Barlenth Hertha Hertling-8	64 54 01	
77 46 71	Barlenth Hertha Hertling-8	64 54 01	
1 59 54 09	Barlenth Hertha Hertling-8	64 54 01	
69 37 89 79	Barlenth Hertha Hertling-8	64 54 01	
6 92 99 56	Barlenth Hertha Hertling-8	64 54 01	
29 91 72	Barlenth Hertha Hertling-8	64 54 01	
22 21 07	Barlenth Hertha Hertling-8	64 54 01	
7 60 50 77	Barlenth Hertha Hertling-8	64 54 01	
6 12 54 82	Barlenth Hertha Hertling-8	64 54 01	
4 48 75 71	Barlenth Hertha Hertling-8	64 54 01	
69 38 63 64	Barlenth Hertha Hertling-8	64 54 01	
78 70 78 40	Barlenth Hertha Hertling-8	64 54 01	
78 93 35	Barlenth Hertha Hertling-8	64 54 01	
6 70 86 47	Barlenth Hertha Hertling-8	64 54 01	
21 96 16 17	Barlenth Hertha Hertling-8	64 54 01	
70 07 68 52	Barlenth Hertha Hertling-8	64 54 01	
3 17 30 23	Barlenth Hertha Hertling-8	64 54 01	
89 16 07 90	Barlenth Hertha Hertling-8	64 54 01	
5 80 23 01	Barlenth Hertha Hertling-8	64 54 01	
7 91 47 67	Barlenth Hertha Hertling-8	64 54 01	
60 66 96 52	Barlenth Hertha Hertling-8	64 54 01	
49 21 05	Barlenth Hertha Hertling-8	64 54 01	
4 31 61 83	Barlenth Hertha Hertling-8	64 54 01	
4 30 15 33	Barlenth Hertha Hertling-8	64 54 01	
29 86 10	Barlenth Hertha Hertling-8	64 54 01	
0172 8 55 18 77	Barlenth Hertha Hertling-8	64 54 01	
26 30 30	Barlenth Hertha Hertling-8	64 54 01	
23 24 14 87	Barlenth Hertha Hertling-8	64 54 01	
12 71 19 35	Barlenth Hertha Hertling-8	64 54 01	
0172 8 56 58 80	Barlenth Hertha Hertling-8	64 54 01	
79 51 09	Barlenth Hertha Hertling-8	64 54 01	
95 41 06 - 0	Barlenth Hertha Hertling-8	64 54 01	
95 41 06 - 1 11	Barlenth Hertha Hertling-8	64 54 01	
4 51 09 63 - 0	Barlenth Hertha Hertling-8	64 54 01	

Телефонна книга, прізвища хаотично розташовані

Jackson, Bill.....	123987152	Gretchen, Marie.....	175827348
Watt, Berny.....	861236192	Innes, Marston.....	968273853
Grünberg, Tolly.....	825391547	Jacques, Amaro.....	948271482
Devereux, Jingyi.....	884026402	Dickens, Azaria	386273869
Raith, Haribert.....	634263782	Kennard, Jean-Christophe.....	968237158
Shafir, Herberto.....	837128238	McCabe, Mabella.....	859672813
Song, Evaristo.....	957278354	Adélar, Wells.....	124817258
Yu, Nora.....	934827628	Pawlitzi, Stephani.....	274915832
Kanzaki, Elvira	301385859	Gabrielson, Salomé.....	491828592
Kazloř, Madelief.....	582375829	Stueck, Alma.....	290571952
Illés, Hildegard.....	869583752	Ayden, Jónás.....	767382951
Hölzer, Marceline.....	195736823	Barret, Everard.....	582871960
Ayers, Timéo.....	239683275	Donaldson, Monique.....	281058673
Blain, Diego.....	482062839	Fidel, Niki.....	483593284
Hillam, Kaylin.....	239518583	Gerig, Laurence.....	581902503

Телефонна книга, прізвища відсортовані

Adélar, Wells.....	124817258	Illés, Hildegard.....	869583752
Ayden, Jónás.....	767382951	Innes, Marston.....	968273853
Ayers, Timéo.....	239683275	Jackson, Bill.....	123987152
Barret, Everard.....	582871960	Jacques, Amaro.....	948271482
Blain, Diego.....	482062839	Kanzaki, Elvira	301385859
Dickens, Azaria	386273869	Kazloř, Madelief.....	582375829
Devereux, Jingyi.....	884026402	Kennard, Jean-Christophe.....	968237158
Donaldson, Monique.....	281058673	McCabe, Mabella.....	859672813
Fidel, Niki.....	483593284	Pawlitzi, Stephani.....	274915832
Gabrielson, Salomé.....	491828592	Raith, Haribert.....	634263782
Gerig, Laurence.....	581902503	Shafir, Herberto.....	837128238
Gretchen, Marie.....	175827348	Stueck, Alma.....	290571952
Grünberg, Tolly.....	825391547	Song, Evaristo.....	957278354
Hillam, Kaylin.....	239518583	Watt, Berny.....	861236192
Hölzer, Marceline.....	195736823	Yu, Nora.....	934827628

Індекси в базах даних



Індекси в базах даних — це спеціальні структури, які покращують швидкість виконання запитів. Вони працюють подібно до індексу в книзі, дозволяючи швидко знаходити потрібні дані без необхідності переглядати всю базу даних.

Сканування БД для пошуку даних є дуже неефективним і вимагає від MongoDB обробки великого обсягу даних.

Переваги:

Швидкість: Значно прискорюють виконання запитів.

Ефективність: Зменшують кількість рядків, які потрібно перевірити для виконання запиту.

Недоліки:

Накладні витрати на запис: Підтримка індексів додає накладні витрати на операції запису (близько 10%).

Використання пам'яті: Індекси займають додатковий простір у пам'яті.

Індекси в MongoDB – Типи індексів

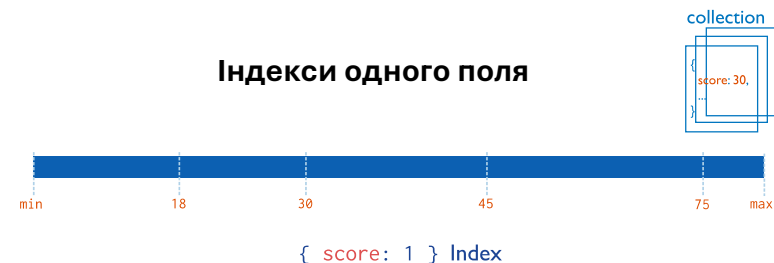
Індекси зберігають невелику частину набору даних у формі, яку легко переглядати.

Індекс зберігає значення певного поля або набору полів, упорядкованих за значенням поля.

У MongoDB існує кілька типів індексів. Деякі із них:

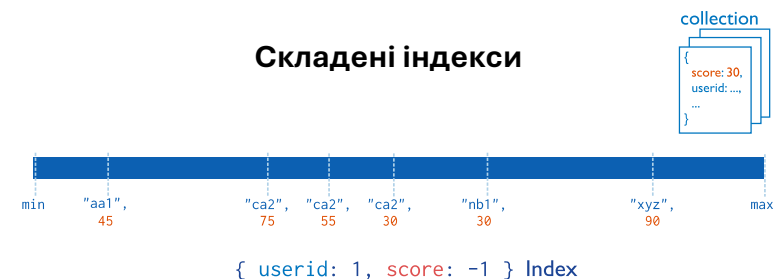
Індекси одного поля (Single Field Index):

Індекс для одного конкретного поля.



Складені індекси (Compound Index):

Індекси для комбінації кількох полів.



Багатоключові індекси (Multikey Index):

Індекси для масивів, де кожен елемент масиву індексується окремо.

Текстові індекси (Text Index):

Індекси для пошуку в текстових полях.

Геопросторовий індекс (Geospatial Index):

Індекси для геопросторових даних (2d indexes та 2dsphere indexes).

Хешовані індекси (Hashed Index):

Індекси, які використовують хешування для зменшення розміру індексу.



Кожен тип індексу має свої особливості та застосування.

Індекси в MongoDB – створення та видалення

Щоб створити індекс у MongoDB, використовуйте метод **createIndex()**.

Створення індексу на полі "name" у колекції "authors":

```
db.authors.createIndex({ name: 1 });
```

"name" - поле, на якому створюється індекс,

"1" - вказує на зростаючий порядок сортування (від А до Z для рядків або від найменшого до найбільшого для чисел). Якщо "-1", це означало б спадний порядок сортування.

Пам'ятайте, що індекси слід створювати лише для полів, які часто використовуються в запитах, щоб забезпечити ефективний доступ до даних. Це допомагає оптимізувати виконання запитів, зменшуючи кількість даних, які потрібно переглянути під час пошуку.

Видалення конкретного індексу з колекції: **dropIndex()**.

```
db.authors.dropIndex({ name: 1 });
```

Видалення всіх індексів з колекції або масив індексів: **db.collection.dropIndexes()**.

```
db.authors.dropIndexes();
```

Індекси в MongoDB – текстові індекси

Використання текстових індексів для пошуку документів за текстовими полями.

```
db.articles.createIndex({ content: "text" });
```

```
db.articles.find({ $text: { $search: "MongoDB" } });
```

Індекси в MongoDB – геопросторові індекси

Використання геопросторових індексів для пошуку документів на основі географічних координат.

```
db.places.createIndex({ location: "2dsphere" });
```

```
{  
  "name": "Central Park",  
  "location": {  
    "type": "Point",  
    "coordinates": [ -73.9654, 40.7829 ]  
  }  
}
```

```
db.places.find({  
  location: {  
    $near: {  
      $geometry: { type: "Point", coordinates: [ -73.9667, 40.78 ] },  
      $maxDistance: 1000  
    }  
  }  
});
```

Створення геопросторового індексу:

Індекс типу **2dsphere** використовується для роботи з даними у форматі **GeoJSON** і підтримує запити, які враховують кривизну Землі.

Вставка документів з геопросторовими даними:

Поле **location** містить координати у форматі **GeoJSON**, де **"type": "Point"** вказує на тип геометрії, а **"coordinates"** містить масив з довготою і широтою.

Виконання геопросторового запиту:

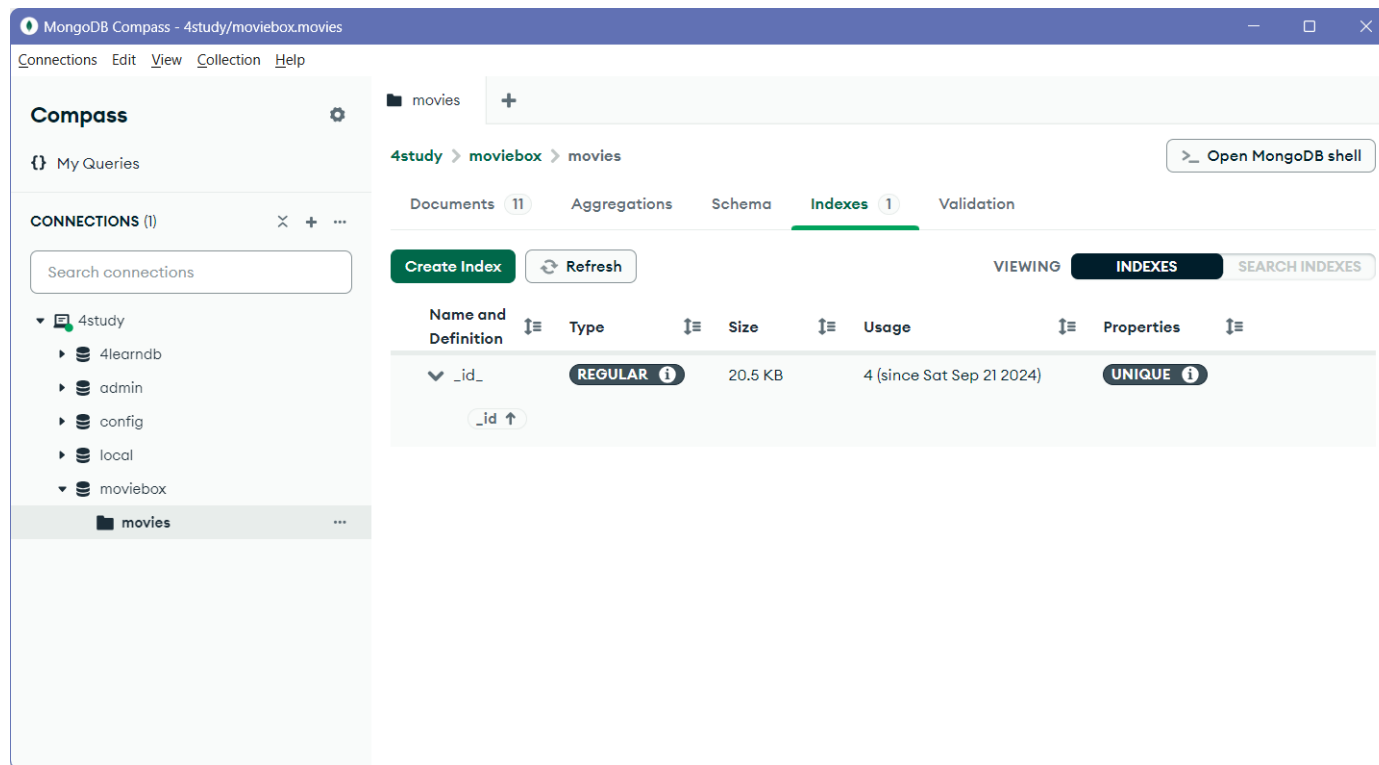
Оператор **\$near** використовується для пошуку документів, які знаходяться поблизу заданої точки. Параметр **\$geometry** визначає тип і координати точки, а **\$maxDistance** задає максимальну відстань у метрах.

Індекс _id

За замовчуванням MongoDB створює унікальний індекс на полі _id під час створення колекції.

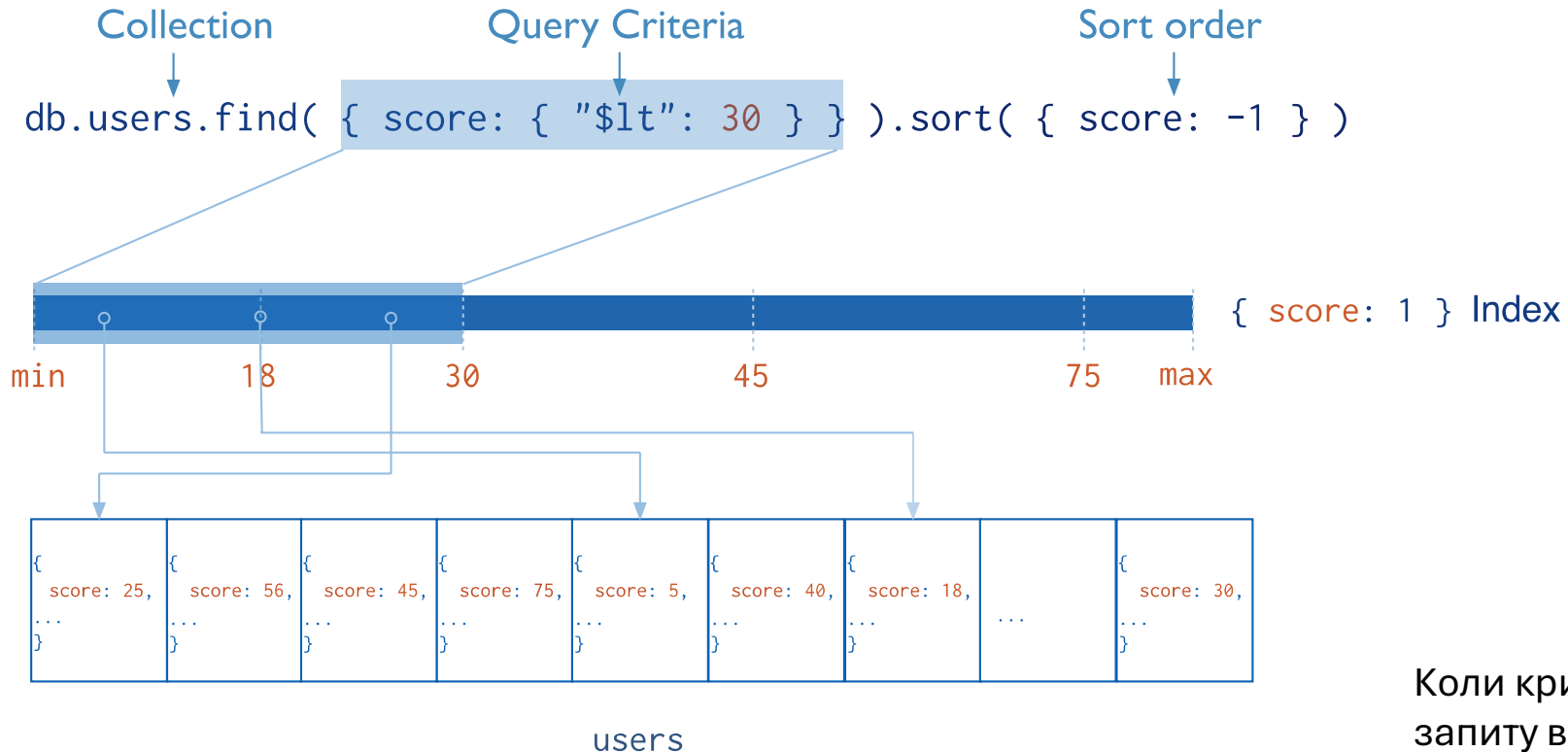
Індекс _id запобігає вставці двох документів з однаковим значенням поля _id.

Ви не можете видалити цей індекс на полі _id.



Індекси в MongoDB

На діаграмі показано запит, який вибирає та впорядковує відповідні документи за допомогою індексу.



Коли критерії запити та проекція (сортуння) запити включають лише індексовані поля, MongoDB повертає результати безпосередньо з індексу, не скануючи жодних документів і не завантажуючи їх у пам'ять. Такі покриті запити (Covered Query) можуть бути дуже ефективними.

Типи даних в MongoDB

Типи даних

Типи даних є фундаментальним аспектом будь-якої системи зберігання та обробки даних.

Вони виконують кілька важливих функцій:

1. Визначення структури даних

Типи даних визначають, які значення можуть бути збережені в полі. Наприклад, поле з типом **String** може містити лише текстові дані, тоді як поле з типом **Integer** може містити лише цілі числа.

2. Ефективне зберігання

Різні типи даних займають різну кількість пам'яті. Використання відповідних типів даних дозволяє оптимізувати використання пам'яті. Наприклад, 32-бітове ціле число займає менше пам'яті, ніж 64-бітове.

3. Валідація даних

Типи даних допомагають забезпечити цілісність даних, перевіряючи, що значення, які зберігаються в полі, відповідають очікуваному типу. Це зменшує ймовірність помилок і забезпечує коректність даних.

4. Оптимізація запитів

Знання типів даних дозволяє системі бази даних оптимізувати виконання запитів. Наприклад, порівняння чисел виконується швидше, ніж порівняння рядків.

5. Підтримка різноманітних операцій

Різні типи даних підтримують різні операції. Наприклад, над числами можна виконувати арифметичні операції, над рядками — операції конкатенації, а над датами — операції порівняння.

6. Сумісність з мовами програмування

Типи даних у базі даних часто відповідають типам даних у мовах програмування, що полегшує інтеграцію та обробку даних у додатках.

7. Забезпечення гнучкості

Підтримка різноманітних типів даних дозволяє зберігати складні структури даних, такі як вкладені документи, масиви, двійкові дані тощо. Це забезпечує гнучкість у моделюванні даних.

8. Підтримка специфічних функцій

Деякі типи даних, такі як **ObjectId** у MongoDB, мають спеціальні функції. **ObjectId** використовується як унікальний ідентифікатор документа і містить інформацію про час створення документа.

BSON типи даних

Type	Number	Alias	Notes
Double	1	"double"	64-бітове число з плаваючою комою
String	2	"string"	Рядок UTF-8
Object	3	"object"	Вкладений документ, який сам по собі є колекцією пар "ключ-значення"
Array	4	"array"	Масив значень
Binary data	5	"binData"	Двійкові дані
Undefined	6	"undefined"	Deprecated
ObjectId	7	"objectId"	Унікальний ідентифікатор документа
Boolean	8	"bool"	Логічне значення (true або false)
Date	9	"date"	Дата і час у мілісекундах з 1 січня 1970 року (Unix epoch)
Null	10	"null"	Значення null
Regular Expression	11	"regex"	Регулярний вираз
DBPointer	12	"dbPointer"	Deprecated
JavaScript	13	"javascript"	Вбудований JavaScript-код
Symbol	14	"symbol"	Deprecated
32-bit integer	16	"int"	32-бітове ціле число
Timestamp	17	"timestamp"	Позначка часу, яка використовується для внутрішніх цілей MongoDB
64-bit integer	18	"long"	64-бітове ціле число
Decimal128	19	"decimal"	128-бітове число з плаваючою комою, яке забезпечує високу точність
Min key	-1	"minKey"	Спеціальний тип, який порівнюється як менший за всі інші значення BSON
Max key	127	"maxKey"	Спеціальний тип, який порівнюється як більший за всі інші значення BSON

Приклад JSON-документа з різними типами даних

Type	Number	Alias
Double	1	"double"
String	2	"string"
Object	3	"object"
Array	4	"array"
Binary data	5	"binData"
Undefined	6	"undefined"
ObjectId	7	"objectId"
Boolean	8	"bool"
Date	9	"date"
Null	10	"null"
Regular Expression	11	"regex"
DBPointer	12	"dbPointer"
JavaScript	13	"javascript"
Symbol	14	"symbol"
32-bit integer	16	"int"
Timestamp	17	"timestamp"
64-bit integer	18	"long"
Decimal128	19	"decimal"
Min key	-1	"minKey"
Max key	127	"maxKey"

```
{
  "_id": ObjectId("507f1f77bcf86cd799439011"), // ObjectId (7)
  "name": "John Doe", // String (2)
  "age": 29, // 32-bit integer (16)
  "isActive": true, // Boolean (8)
  "balance": Decimal128("1234.56"), // Decimal128 (19)
  "createdAt": ISODate("2023-10-01T00:00:00Z"), // Date (9)
  "tags": ["developer", "mongodb"], // Array (4)
  "address": { // Object (3)
    "street": "123 Main St", // String (2)
    "city": "Anytown", // String (2)
    "zip": "12345" // String (2)
  },
  "profilePicture": BinData(0, "iVBORw0KGgoAAAANSUUhEUgAAAAUA"), // Binary data (5)
  "lastLogin": Timestamp(1633024800, 1), // Timestamp (17)
  "preferences": { // Object (3)
    "notifications": true, // Boolean (8)
    "theme": "dark" // String (2)
  },
  "comment": { // Object (3)
    "date": ISODate("2023-09-30T12:34:56Z"), // Date (9)
    "text": "This is a comment.", // String (2)
    "likes": 10 // 32-bit integer (16)
  },
  "customData": { // Object (3)
    "type": "Buffer", // String (2)
    "data": [1, 2, 3, 4, 5] // Array (4)
  },
  "regexExample": /pattern/i, // Regular Expression (11)
  "jsCode": function() { return "Hello, World!"; }, // JavaScript (13)
  "minValue": MinKey(), // Min key (-1)
  "maxValue": MaxKey(), // Max key (127)
  "longNumber": NumberLong("9223372036854775807"), // 64-bit integer (18)
  "doubleValue": 123.456, // Double (1)
  "nullValue": null // Null (10)
}
```

BSON типи даних – приклад типу та числового коду

Type	Number	Alias
Double	1	"double"
String	2	"string"
Object	3	"object"
Array	4	"array"
Binary data	5	"binData"
Undefined	6	"undefined"
ObjectId	7	"objectId"
Boolean	8	"bool"
Date	9	"date"
Null	10	"null"
Regular Expression	11	"regex"
DBPointer	12	"dbPointer"
JavaScript	13	"javascript"
Symbol	14	"symbol"
32-bit integer	16	"int"
Timestamp	17	"timestamp"
64-bit integer	18	"long"
Decimal128	19	"decimal"
Min key	-1	"minKey"
Max key	127	"maxKey"

```
{
  "_id": ObjectId("507f1f77bcf86cd799439011"),
  "name": "John Doe",
  "age": 29,
  "isActive": true,
  "balance": Decimal128("1234.56"),
  "createdAt": ISODate("2023-10-01T00:00:00Z")
}
```

```
\x07 _id \x00 \x12 507f1f77bcf86cd799439011
\x02 name \x00 \x08 John Doe \x00
\x10 age \x00 \x1d \x00 \x00 \x00
\x08 isActive \x00 \x01
\x13 balance \x00 \x12 1234.56
\x09 createdAt \x00 \x12 2023-10-01T00:00:00Z
```

- \x07 — числовий код для ObjectId.
- \x02 — числовий код для String.
- \x10 — числовий код для 32-bit integer.
- \x08 — числовий код для Boolean.
- \x13 — числовий код для Decimal128.
- \x09 — числовий код для Date.

Числові коди не використовуються безпосередньо в запитах або документах, які ви пишете, вони є важливою частиною внутрішньої архітектури MongoDB.

Агрегація в MongoDB

Агрегація

Агрегація — це процес об'єднання, зведення або узагальнення даних для отримання підсумкової інформації.

Вона використовується для виконання різних операцій над даними, таких як підрахунок, обчислення середніх значень, сума, мінімум, максимум, групування та інші статистичні або аналітичні операції.

Агрегація дозволяє отримати зведену інформацію з великих наборів даних, що допомагає в аналізі та прийнятті рішень.

У контексті баз даних, агрегація часто реалізується за допомогою спеціальних функцій і операторів, які дозволяють виконувати складні операції над даними.

Наприклад, у SQL базах даних використовуються функції SUM(), AVG(), COUNT(), MIN(), MAX(), а також оператори GROUP BY для групування даних.



Агрегація – основні аспекти

Групування даних

Дані можуть бути згруповані за певними критеріями або полями. Наприклад, можна згрупувати продажі за місяцями або клієнтами.

Обчислення підсумкових значень

Після групування можна обчислити підсумкові значення, такі як сума продажів, середнє значення, мінімум або максимум.

Фільтрація даних

Перед агрегацією дані можуть бути відфільтровані за певними критеріями, щоб включити в обробку лише релевантні записи.

Сортування даних

Результати агрегації можуть бути відсортовані за певними полями для зручності аналізу.

Приклади агрегації

- Підрахунок кількості продажів за кожен місяць.
- Обчислення середньої ціни товарів у кожній категорії.
- Обчислення загальної суми продажів за кожного клієнта.
- Знаходження мінімальної та максимальної зарплати серед працівників.

Агрегація в MongoDB

Агрегація в MongoDB — це потужний інструмент для обробки та аналізу даних, що дозволяє виконувати складні операції на колекціях документів.



У MongoDB агрегація реалізована через

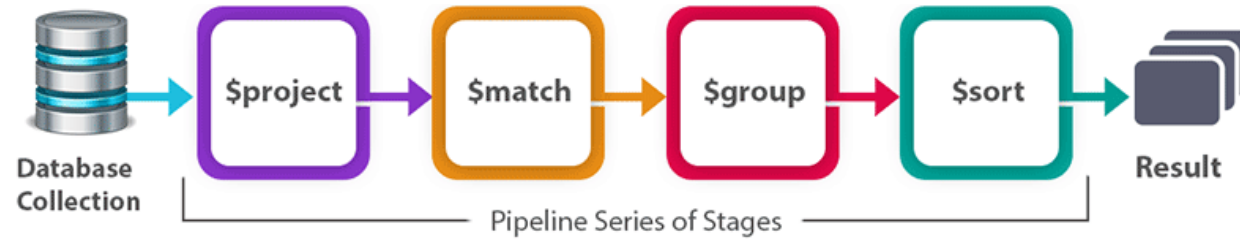
агрегаційний фреймворк, який використовує конвеєр (pipeline) для обробки даних.

Кожна стадія конвеєра виконує певну операцію над документами і передає результати наступній стадії.

Вона дозволяє виконувати такі завдання, як фільтрація, сортування, групування, обчислення підсумкових значень та багато іншого.

`db.COLLECTION_NAME.aggregate(AGGREGATE_OPERATION)`

Основні поняття та компоненти агрегації в MongoDB



1.Пайплайн (Pipeline).

Агрегаційний пайплайн складається з послідовності стадій, через які проходять документи. Кожна стадія виконує певну операцію над документами і передає результати наступній стадії.

2.Стадії (Stages).

Кожна стадія в пайплайні виконує певну операцію. Деякі з найпоширеніших стадій:

- **\$project**: Вибирає певні поля для включення або виключення з результатів.
- **\$match**: Фільтрує документи на основі певних критеріїв.
- **\$group**: Групує документи за певним полем і дозволяє виконувати агрегатні функції, такі як сума, середнє значення тощо.
- **\$sort**: Сортуює документи.
- **\$limit**: Обмежує кількість документів у результатах.
- **\$skip**: Пропускає певну кількість документів.
- **\$lookup**: Виконує операцію з'єднання (join) з іншою колекцією.

3.Агрегатні функції:

Функції, які використовуються для обчислення підсумкових значень, такі як **\$sum**, **\$avg**, **\$min**, **\$max**, **\$push**, **\$addToSet** тощо.

Агрегатні функції в MongoDB

Вираз	Опис	Приклад
\$sum	Підсумовує визначене значення з усіх документів у колекції.	<code>db.mycol.aggregate([{\$group : {_id : "\$by_user", num_tutorial : {\$sum : "\$likes"}}}])</code>
\$avg	Обчислює середнє значення всіх заданих значень з усіх документів у колекції.	<code>db.mycol.aggregate([{\$group : {_id : "\$by_user", num_tutorial : {\$avg : "\$likes"}}}])</code>
\$min	Отримує мінімальне значення відповідних значень з усіх документів у колекції.	<code>db.mycol.aggregate([{\$group : {_id : "\$by_user", num_tutorial : {\$min : "\$likes"}}}])</code>
\$max	Отримує максимальне значення відповідних значень з усіх документів у колекції.	<code>db.mycol.aggregate([{\$group : {_id : "\$by_user", num_tutorial : {\$max : "\$likes"}}}])</code>
\$push	Вставляє значення в масив у результативному документі.	<code>db.mycol.aggregate([{\$group : {_id : "\$by_user", url : {\$push : "\$url"}}}])</code>
\$addToSet	Вставляє значення в масив у результативному документі, але не створює дублікатів.	<code>db.mycol.aggregate([{\$group : {_id : "\$by_user", url : {\$addToSet : "\$url"}}}])</code>
\$first	Отримує перший документ з вихідних документів відповідно до групування. Зазвичай це має сенс разом з попередньо застосованою стадією \$sort.	<code>db.mycol.aggregate([{\$group : {_id : "\$by_user", first_url : {\$first : "\$url"}}}])</code>
\$last	Отримує останній документ з вихідних документів відповідно до групування. Зазвичай це має сенс разом з попередньо застосованою стадією \$sort.	<code>db.mycol.aggregate([{\$group : {_id : "\$by_user", last_url : {\$last : "\$url"}}}])</code>

Приклад агрегаційного запиту

`db.COLLECTION_NAME.aggregate(AGGREGATE_OPERATION)`

`db. COLLECTION_NAME.aggregate([ {stage1}, {stage2}, {stage3}...])`

```
db.orders.aggregate([
  { $match: { status: "completed" } }, // Фільтруємо документи зі статусом "completed"
  { $group: { _id: "$customerId", totalAmount: { $sum: "$amount" } } }, // Групуємо за customerId і обчислюємо загальну суму
  { $sort: { totalAmount: -1 } } // Сортуємо за загальною сумою у спадному порядку
]);
```

У цьому прикладі:

`$match` фільтрує документи, залишаючи тільки ті, що мають статус `"completed"`.

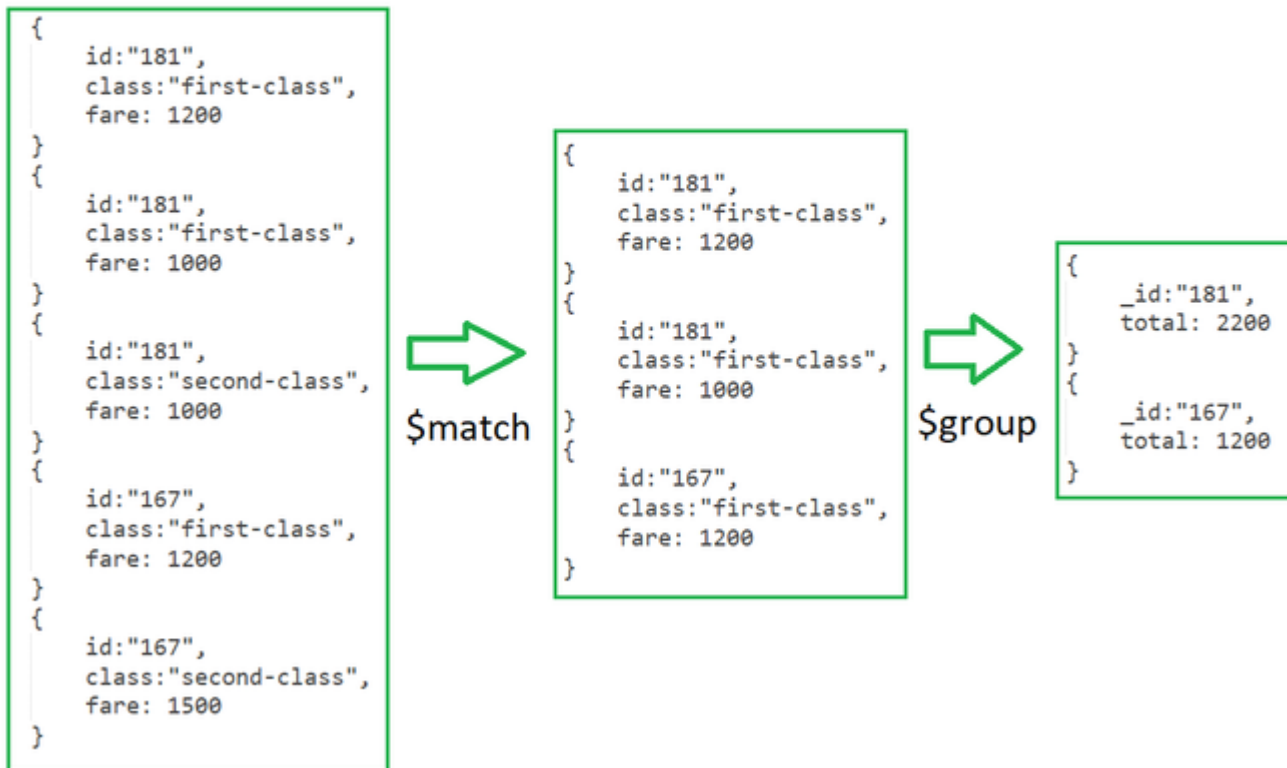
`$group` групує документи за полем `customerId` і обчислює загальну суму (`$sum`) для кожного клієнта.

`$sort` сортує результати за полем `totalAmount` у спадному порядку.

Приклад агрегаційного запиту

```
db.train.aggregate( [
  { $match: { class: "first-class" } },
  { $group: { _id: "id", total: { $sum: "$fare" } } }
])
```

} pipeline stages



Поле `_id` обов'язкове у стадії `$group` для визначення, як документи будуть групуватися.

Якщо ви хочете зберегти результати агрегації в новій колекції, остання стадія пайплайну повинна бути `$out`.

Оператор `$sum` використовується для підрахунку кількості документів або обчислення суми значень певного поля.

Використання `$sum` з полем `$fare` обчислює загальну суму тарифу для кожного ID.

Приклад агрегаційного запиту

Collection
↓
`db.orders.aggregate( [`
 `$match stage → { $match: { status: "A" } },`
 `$group stage → { $group: { _id: "$cust_id", total: { $sum: "$amount" } } }`
 `] )`

{ cust_id: "A123", amount: 500, status: "A" }
{ cust_id: "A123", amount: 250, status: "A" }
{ cust_id: "B212", amount: 200, status: "A" }
{ cust_id: "A123", amount: 300, status: "D" }

orders

`$match` →

{ cust_id: "A123", amount: 500, status: "A" }
{ cust_id: "A123", amount: 250, status: "A" }
{ cust_id: "B212", amount: 200, status: "A" }

`$group` →

Results	
{	<code>_id: "A123",</code>
	<code>total: 750</code>
}	
{	<code>_id: "B212",</code>
	<code>total: 200</code>
}	

Переваги агрегації

Гнучкість

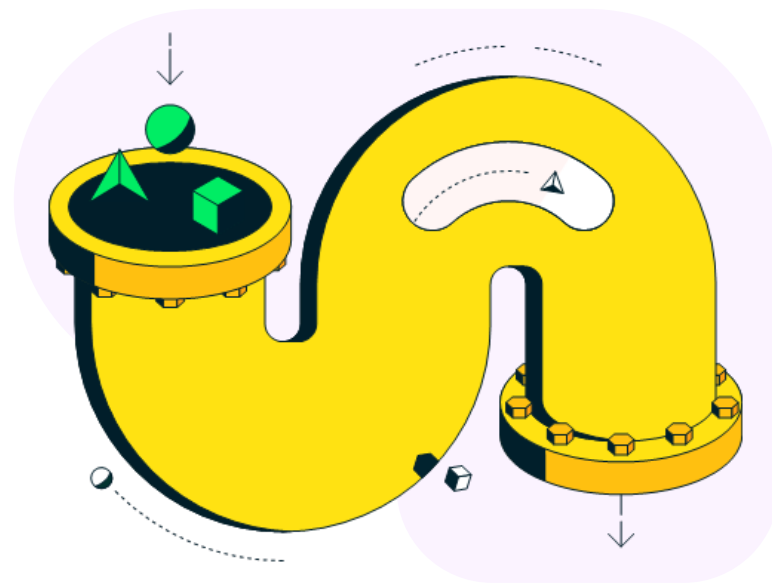
Агрегаційний фреймворк дозволяє виконувати складні операції на даних, включаючи фільтрацію, групування, сортування та обчислення підсумкових значень.

Продуктивність

Агрегаційні операції виконуються на сервері MongoDB, що дозволяє обробляти великі обсяги даних ефективніше, ніж витягувати всі документи на клієнтську сторону для обробки.

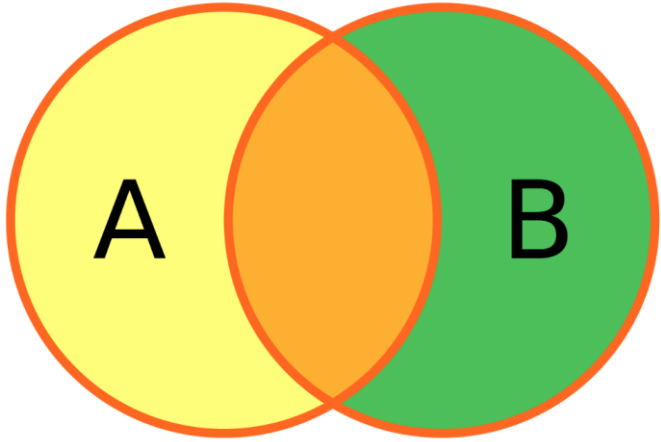
Масштабованість

Агрегаційний фреймворк добре масштабується з великими обсягами даних і може використовувати індекси для оптимізації виконання запитів.



Пов'язування документів в MongoDB

Пов'язування документів



Пов'язування документів (або "Joins") — це процес об'єднання даних з різних документів або колекцій на основі певних умов.

У реляційних базах даних це зазвичай досягається за допомогою операцій JOIN, які об'єднують рядки з двох або більше таблиць на основі спільного поля.

MongoDB є документно-орієнтованою базою даних, і вона не підтримує класичні SQL-запити для об'єднання таблиць. Тому підхід до пов'язування документів дещо відрізняється.

Є кілька способів пов'язування документів в MongoDB:

- Вбудовані документи (Embedded Documents)
- Посилання (References)
- Операція \$lookup

Пов'язування документів – Вбудовані документи

Вбудовані документи (Embedded Documents)

Вкладені документи дозволяють зберігати пов'язані дані в одному документі. Це зручно для даних, які часто використовуються разом.

```
{
  "_id": 1,
  "name": "John Doe",
  "address": {
    "street": "123 Main St",
    "city": "Anytown",
    "zip": "12345"
  }
}
```

Переваги: Швидкий доступ до пов'язаних даних, оскільки вони зберігаються в одному документі.

Недоліки: Може призвести до дублювання даних і збільшення розміру документів.

Пов'язування документів – Посилання

Посилання (References)

Посилання використовуються для зберігання відносин між документами в різних колекціях. Це схоже на зовнішні ключі в реляційних базах даних.

```
{
  "_id": 1,
  "name": "John Doe",
  "address_id": 101
}
```

```
{
  "_id": 101,
  "street": "123 Main St",
  "city": "Anytown",
  "zip": "12345"
}
```

Переваги: Зменшує дублювання даних і дозволяє зберігати великі обсяги пов'язаних даних.

Недоліки: Потребує додаткових запитів для отримання пов'язаних даних.

Пов'язування документів – Операція \$lookup

Операція \$lookup

Операція \$lookup в агрегаційному фреймворку MongoDB дозволяє виконувати з'єднання (join) між колекціями.

```
db.orders.aggregate([
  {
    $lookup: {
      from: "customers",
      localField: "customerId",
      foreignField: "_id",
      as: "customerDetails"
    }
  }
]);
```

Переваги: Дозволяє виконувати складні з'єднання між колекціями без необхідності додаткових запитів.

Недоліки: Може бути менш ефективним для дуже великих наборів даних.

Пов'язування документів – Коли використовувати кожен метод

Вбудовані документи:

Використовуйте для часто використовуваних разом і невеликих за обсягом даних. Забезпечує швидкий доступ до всіх даних в одному запиті.

Посилання:

Використовуйте для великих або окремо використовуваних даних. Зменшує дублювання і забезпечує гнучкість структури даних.

Операція \$lookup:

Використовуйте для складних з'єднань між колекціями. Дозволяє отримати пов'язані дані в одному запиті, але може бути менш ефективним для великих наборів даних.

Ad-hoc запити в MongoDB

Ad-hoc запити (Ad-hoc queries)

Ad-hoc запити (Ad-hoc queries) — це запити, які створюються та виконуються на льоту для отримання конкретної інформації з бази даних.

Вони не є заздалегідь визначеними або збереженими, а формуються відповідно до поточних потреб користувача або додатку.

Ad-hoc запити дозволяють гнучко та динамічно взаємодіяти з даними, що є однією з ключових особливостей MongoDB.

Основні характеристики Ad-hoc запитів

Гнучкість

Ad-hoc запити дозволяють користувачам формувати запити на основі поточних потреб, без необхідності заздалегідь визначати структуру запитів.

Динамічність

Запити можуть бути змінені або створені в будь-який момент, що дозволяє швидко адаптуватися до змін у вимогах або даних.

Використання індексів

MongoDB автоматично використовує індекси для оптимізації виконання ad-hoc запитів, що забезпечує високу продуктивність навіть для складних запитів.

Підтримка різних операторів

MongoDB підтримує широкий набір операторів для фільтрації, сортування, проєкції та агрегації даних, що дозволяє створювати складні та потужні запити.

Приклади Ad-hoc запитів у MongoDB

Простий запит:

Знайти всі документи в колекції users, де поле age більше 25.

```
db.users.find({ age: { $gt: 25 } });
```

Запит з проекцією:

Знайти всі документи в колекції users, де поле age більше 25, і повернути тільки поля name та age.

```
db.users.find({ age: { $gt: 25 } }, { name: 1, age: 1 });
```

Запит з сортуванням:

Знайти всі документи в колекції users, де поле age більше 25, і відсортувати результати за полем name у зростаючому порядку.

```
db.users.find({ age: { $gt: 25 } }).sort({ name: 1 });
```

Запит з обмеженням кількості результатів:

Знайти всі документи в колекції users, де поле age більше 25, і повернути тільки перші 10 результатів.

```
db.users.find({ age: { $gt: 25 } }).limit(10);
```

Запит з використанням регулярних виразів:

Знайти всі документи в колекції users, де поле name починається з літери "J".

```
db.users.find({ name: { $regex: /^J/ } });
```

Переваги Ad-hoc запитів у MongoDB

Швидкість розробки:

Можливість швидко створювати та виконувати запити без необхідності заздалегідь визначати їх структуру.

Гнучкість:

Запити можуть бути адаптовані до змін у вимогах або даних, що дозволяє швидко реагувати на нові запити від користувачів або додатків.

Продуктивність:

Використання індексів та оптимізацій MongoDB забезпечує високу продуктивність навіть для складних ad-hoc запитів.



ДЯКУЮ ЗА УВАГУ!